

INFORME LABORATORIO 1 COMPUTACIÓN PARALELA

Cremona Javier
Castillo Marcelo

Presentación: <https://youtu.be/oZRH8gufSP4>

Introducción

La técnica de Monte Carlo es un método flexible que permite simular la propagación de la luz en los tejidos. La simulación se basa en las trayectorias aleatorias que realizan los fotones a medida que viajan a través del tejido, que se eligen muestreando estadísticamente las distribuciones de probabilidad para el tamaño del paso y la deflexión angular por evento de dispersión. Después de propagar muchos fotones, la distribución neta de todas las trayectorias de fotones produce una aproximación precisa a la realidad.(1)

Hay una variedad de formas de implementar simulaciones Monte Carlo de transporte ligero. Un enfoque es predecir distribuciones de luz en estado estable. El otro enfoque es predecir distribuciones de luz resueltas en el tiempo.

Boost es un conjunto de bibliotecas de software libre y revisión por pares preparadas para extender las capacidades del lenguaje de programación C++. Esta biblioteca proporciona varios generadores de números pseudoaleatorios. La calidad de un generador de números pseudoaleatorios depende de manera crucial tanto del algoritmo como de sus parámetros. Esta biblioteca implementa los algoritmos como plantillas de clase con parámetros de valor de plantilla, ocultos en el espacio de nombres boost::random. Esto permite escribir código en el que se puede cambiar fácilmente de un algoritmo a otro..

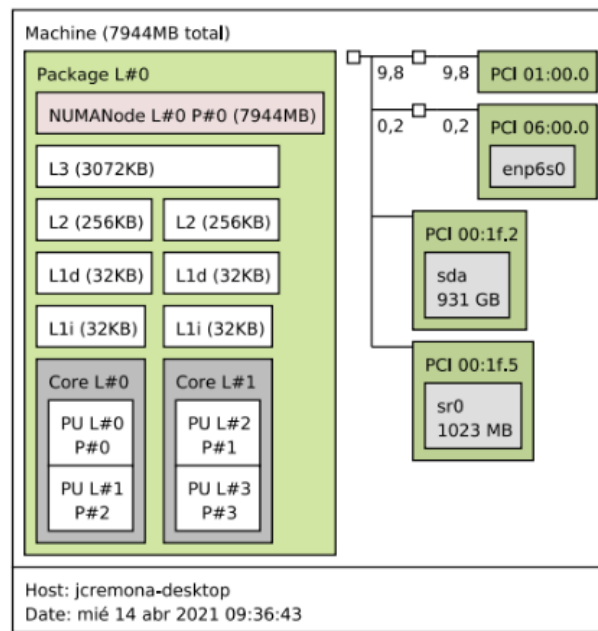
Los generadores de números pseudoaleatorios tienen sus particularidades. No deben inicializarse con frecuencia durante la ejecución del programa, pues suelen ser costosos de inicializar. Adicionalmente, la inicialización siempre requiere que se utilice algún valor como "semilla" para la secuencia generada. Por lo general, es difícil obtener varios valores de semillas buenos. Por ejemplo, un método para obtener una semilla es determinar la hora actual con la resolución más alta disponible, por ejemplo microsegundos o nanosegundos. Cuando el generador de números pseudoaleatorios se inicializa nuevamente con el tiempo actual en ese momento como semilla, es probable que esté a una distancia casi constante (no aleatoria) del tiempo dado como semilla para la primera inicialización.

En este trabajo se realiza la optimización del programa tiny_mc.c.

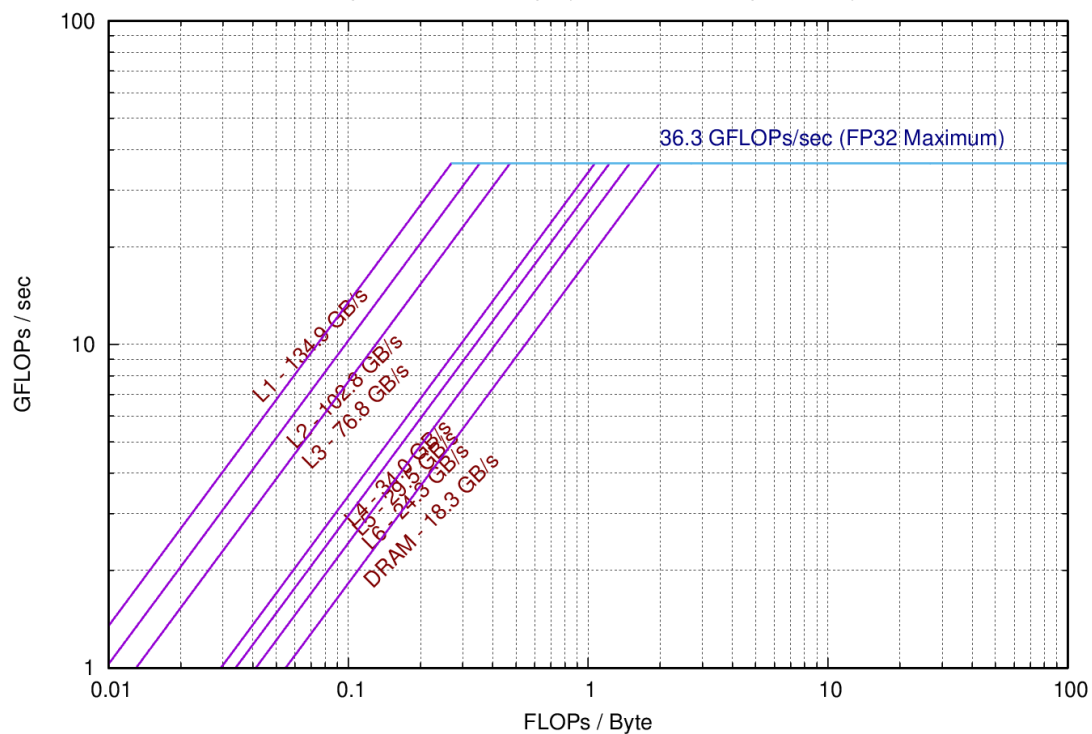
Materiales y Metodología

La computadora empleada para realizar el laboratorio se describe a continuación:

- Características del hardware y del software:
 - CPU: Intel Core i3-2100 CPU @3.10GHz-(Sandy Bridge)
 - RAM: DDR3, 2 canales, tamaño 8 GB, velocidad 1333 MT/s.
Ranura: A0, tamaño: 4GB, anchura : 2244 bits
Ranura: A2, tamaño: 4GB, anchura : 2244 bits
 - Sistema Operativo: Ubuntu 20.04.2 LTS
 - Kernel: Linux 5.8.0-48-generic
 - Arquitectura: x86_64
 - Compiladores: gcc 9.3.0 y clang 10.0.0

Istopo**ERT**

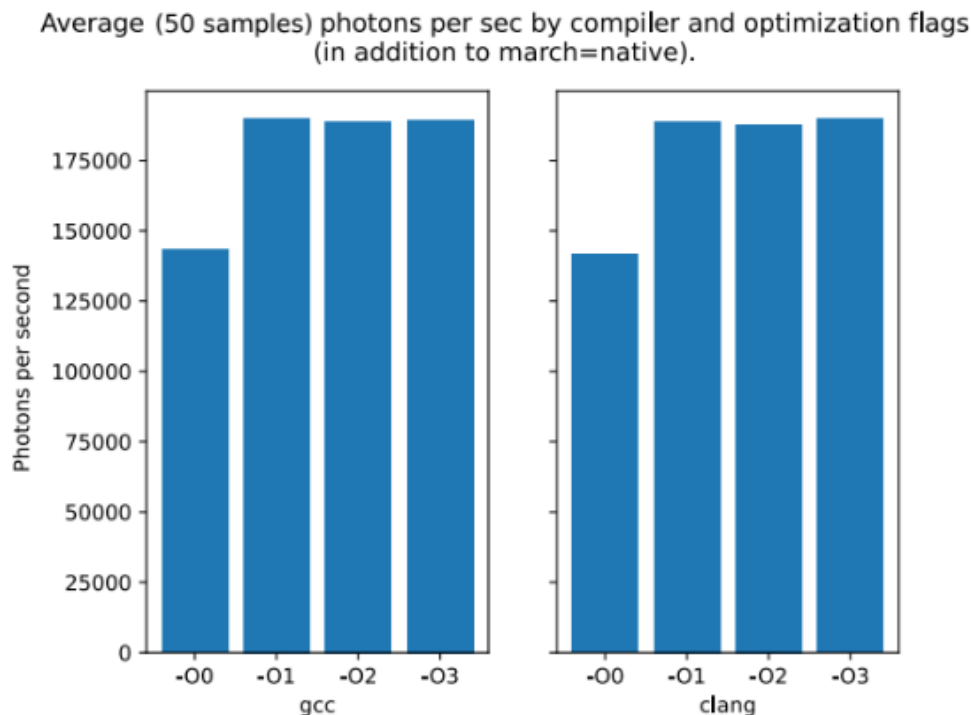
Empirical Roofline Graph (Resultados.desktop/Run.001)



Empleando estos recursos se optimizó el trabajo con flags y a nivel de código. Adicionalmente se implementó código para automatización con Python. La métrica empleada es fotones por segundo (pps), pues el problema es estocástico y no tiene sentido comparar FLOPS. En general, en cada caso se realizaron 50 corridas para eliminar el system noise y se tomó el promedio de estos valores para presentar los resultados.

Flags

Se exploró -O0, -O1, -O2 y -O3 para gcc 9.3 y clang 10.0 (en conjunto con -march=native) sobre el código original.



Vemos que el código original y solo con `-march=native` genera 140K pps. Ese es nuestro resultado base. A partir de aquí se optó por continuar con gcc, `-O3` y `-march=native`. Éstas opciones de compilación son básicas, y entre los dos compiladores pareciera no haber diferencia en el rendimiento obtenido.

Luego de cambiar el generador de números aleatorios (ver Resultados), se probó con `-ffast-math` y `-fltto`. También se probó la técnica FDO y las banderas para optimizar loops.

A nivel de código se realizaron 3 grandes modificaciones:

Primer cambio:

Cambio de lenguaje de C a C++ para aprovechar otras librerías de randomización. Después de un primer profiling con perf, se concluyó que `rand()` de la `stdlib` es un método caro de utilizar para implementar Montecarlo. En particular se utilizó Boost. Se probaron 3 algoritmos de obtención de números pseudoaleatorios: `taus88`, `rand48` y `mt19937` (Mersenne Twister). Luego en el programa se cambian las concurrencias de `rand()` por la llamada al método correspondiente de Boost.

Segundo cambio:

Se movió la condición de corte del bucle del método `photon()`. Esto es para evitar la generación innecesaria de números aleatorios (generaba como mínimo 2 flotantes de más en la última iteración que nunca se iban a usar).

Tercer cambio:

El programa emplea dos arreglos de tamaño constante, `heat` y `heat2`. En cada iteración se accede a la misma posición de memoria del arreglo (por ej, `heat[i]` y `heat2[i]`, para algún índice `i`). Se cambian estos dos arreglos por un solo array de estructura, en el que cada elemento tiene dos flotantes.

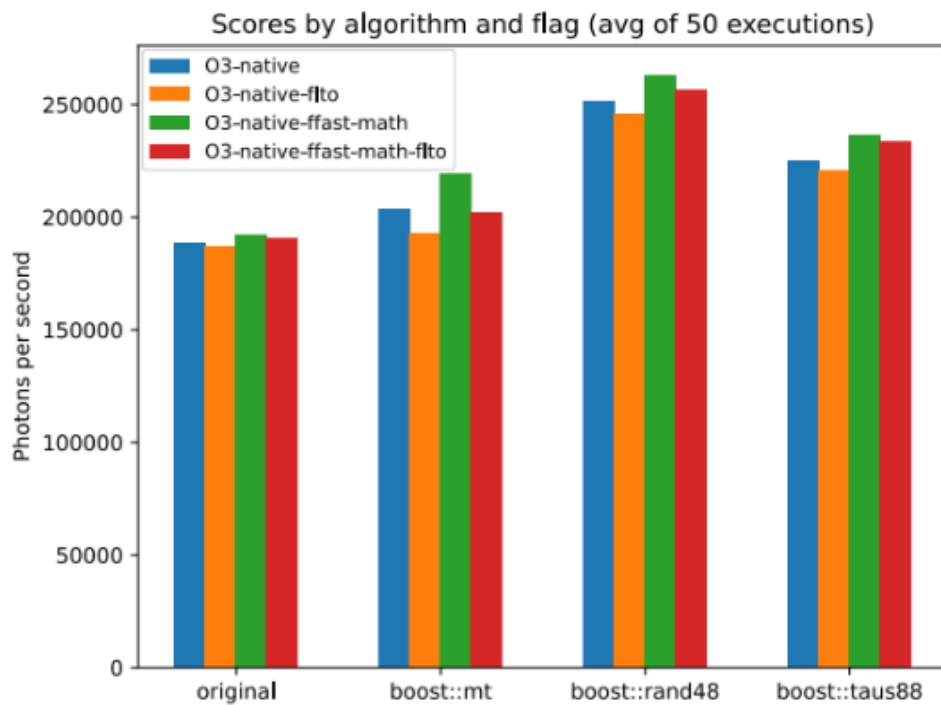
Para más detalle, ver la presentación: <https://youtu.be/oZRH8gufSP4>

Otros cambios fallidos

- Se trabajó en la generación de los números aleatorios que se emplean para calcular la nueva dirección del fotón. Dado que se requieren números dentro de la circunferencia de radio 1 centrada en el origen, se cambió a coordenadas polares, sampleando el radio entre 0 y 1 y un ángulo entre 0 y 2π . Luego, se utilizó seno y coseno para volver a coordenadas cartesianas. Con esta modificación, el rendimiento disminuyó. Se realizó profiling con perf y se encontró que las funciones `sinf()` y `cosf()` de `math.h` son computacionalmente caras para nuestro problema. Por lo tanto se optó por descartar esta opción. Una alternativa sería emplear aproximaciones computacionalmente eficientes para seno y coseno. En general, se pierde precisión con estas aproximaciones.
- Se intentó aplicar optimizaciones de loop en el método `photon()`. Loop unrolling no prosperó ya que nos vimos limitados por la dependencia entre valores de variables entre iteraciones. Loop blocking no tiene sentido en nuestro problema, ya que el arreglo de estructura es accedido en posiciones aleatorias en cada iteración. Por último, se intentó aplicar loop peeling para ahorrar operaciones aritméticas en la primera iteración. Esto trajo una mejora muy pequeña, y con FDO esta mejora era imperceptible. Dado que este cambio resultaba en un código poco mantenible de `photon()`, fue descartado. Evidentemente, el compilador también se vio limitado, ya que utilizar `-funroll-loops` y `-floop-block` no nos mejoró la cantidad de pps obtenidos.

Resultados

- Encontramos que la función `rand` (de la `stdlib`) que permite obtener números aleatorios es computacionalmente cara y hay que evitarla. Por lo tanto se probó con 3 algoritmos de obtención de números pseudoaleatorios implementados en Boost: Mersenne Twister (uno de los más usados algoritmos de PRNG, pero no de los más rápidos), `rand48` (un algoritmo lineal congruencial) y `taus88`. Estos dos últimos se eligieron a partir de consultar la documentación de Boost y ver que eran los más rápidos. Además de las banderas de `-O3` y `-march=native`, se probó con `-ffast-math` y `-flto`. Se encontró con que `-ffast-math` trajo buenos resultados. Además se estudió la salida del programa y se concluyó que esta bandera se puede utilizar de forma segura en este problema por las cantidades numéricas que maneja (órdenes de magnitud no tan extremos). Por el contrario, `-flto` disminuyó el rendimiento obtenido. Esto es porque nuestro programa cuenta con una unidad de compilación (`tiny_mc.cpp`) y no es necesario optimizar entre distintas unidades de compilación. Quizás en tiempo de link no hay optimizaciones que se hacen en tiempo de compilación propiamente dicho. A continuación se muestran los resultados. Se optó por continuar con `-O3`, `-ffast-math` y `-march=native` y `rand48`, lo que nos permitió obtener cerca de 260K pps.



- Al pasar de dos arreglos a uno solo (de estructura) mejora la latencia del programa, es decir, el programa se dirige al índice i -ésimo del arreglo y toma más rápidos los valores al estar contiguos en memoria y por ende cacheados. Vale mencionar que en este programa no aumenta la cantidad de memoria usada por el arreglo al aumentar el número de fotones. Dado que la cantidad de memoria usada por el arreglo es relativamente pequeña (200 floats aproximadamente), es probable que el arreglo completo se mantenga en caché. Sin embargo, encontramos una mejora en el rendimiento con este cambio.

```
for (;;) {
    float t = -logf(rand() / (float)RAND_MAX); /* move */
    x += t * u;
    y += t * v;
    z += t * w;

    unsigned int shell = sqrtf(x * x + y * y + z * z) * shells_per_mfp; /* absorb */
    if (shell > SHELLS - 1) {
        shell = SHELLS - 1;
    }
    heat[shell] += (1.0f - albedo) * weight;
    heat2[shell] += (1.0f - albedo) * (1.0f - albedo) * weight * weight; /* add up squares */
    weight *= albedo;
}
```

Figure: Loop dentro del método photon()

Figure: Código original (izq.) y la modificación aplicada (der.).

```
// global state,
// heat and heat square
// in each shell
static float heat[SHELLS];
static float heat2[SHELLS];
```

```
// global state
struct heat_struct {
    float heat; // heat
    float heat2; // heat square
};

static heat_struct heat_array[SHELLS];
```

- Mover el break: Se movió la condición de corte como se muestra en la siguiente imagen debajo. Antes hacía 2 números flotantes de más que nunca se utilizaban en la última iteración. Al realizar este cambio se observa que la velocidad de ejecución del programa aumentó. Con este y el anterior cambio, se obtuvo una mejora de aproximadamente el 6% en cuanto a los pps obtenidos.

Figure: Código original (izq.) y la modificación aplicada (der.).

```
/* New direction, rejection method */
float x11, x12;
do {
    x11 = 2.0f * rand() / (float)RAND_MAX - 1.0f;
    x12 = 2.0f * rand() / (float)RAND_MAX - 1.0f;
    t = x11 * x11 + x12 * x12;
} while (1.0f < t);
u = 2.0f * t - 1.0f;
v = x11 * sqrtf((1.0f - u * u) / t);
w = x12 * sqrtf((1.0f - u * u) / t);

if (weight < 0.001f) { /* roulette */
    if (rand() / (float)RAND_MAX > 0.1f)
        break;
    weight /= 0.1f;
}
```

```
/* Rejection method */
if (weight < 0.001f) { /* roulette */
    if (random_float() > 0.1f)
        break;
    weight /= 0.1f;
}

/* New direction */
float x11, x12;
do {
    x11 = 2.0f * random_float() - 1.0f;
    x12 = 2.0f * random_float() - 1.0f;
    t = x11 * x11 + x12 * x12;
} while (1.0f < t);
u = 2.0f * t - 1.0f;
v = x11 * sqrtf((1.0f - u * u) / t);
w = x12 * sqrtf((1.0f - u * u) / t);
```

Usando ambas modificaciones se logra una mejora del 6%, llegando a 275K pps.

- Trabajar con coordenadas polares se descartó, de esta experiencia se pudo comprobar que las funciones seno y coseno son costosas, y también lo es log. Se intentó, también, aproximar las funciones $\sin$ y $\cos$ pero la imprecisión nos pareció grande por lo que se descartó esta idea aunque nos sirvió para conocer las limitaciones computacionales de estas funciones.

```
/* New direction */
float x11, x12, ang;
ang = (random_float()) * M_PI * 2;
t = sqrtf(random_float());
x11 = t * cosf(ang);
x12 = t * sinf(ang);

u = 2.0f * t - 1.0f;
v = x11 * sqrtf((1.0f - u * u) / t);
w = x12 * sqrtf((1.0f - u * u) / t);
```

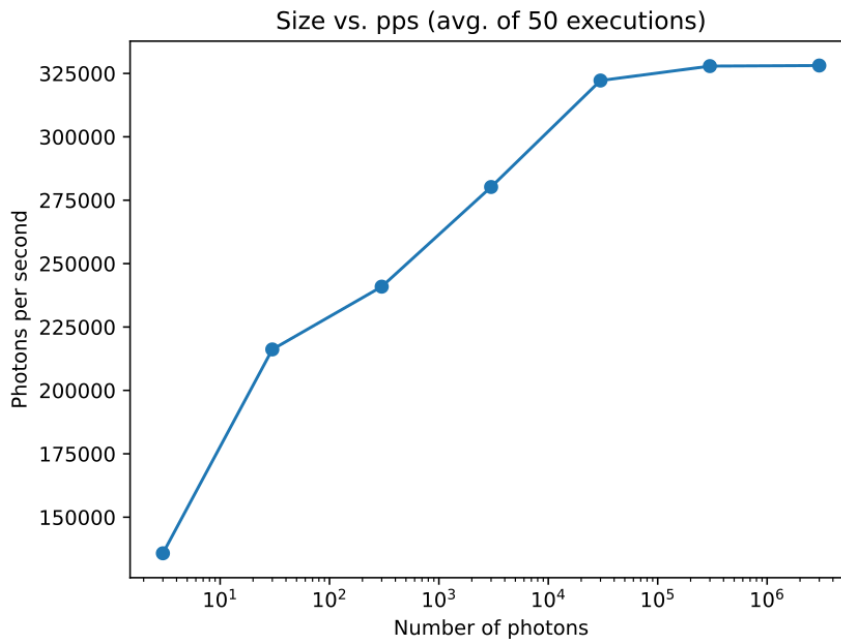
Figure: Código con coordenadas polares.

Samples: 548 of event 'cycles', Event count (approx.): 390088878

Overhead	Command	Shared Object	Symbol
75,57%	tiny_mc	tiny_mc	[.] main
12,91%	tiny_mc	libm-2.31.so	[.] __sincosf_sse2
8,61%	tiny_mc	libm-2.31.so	[.] __logf_sse2
0,78%	tiny_mc	tiny_mc	[.] 0x00000000000011d4
0,39%	tiny_mc	tiny_mc	[.] 0x0000000000001124
0,29%	tiny_mc	tiny_mc	[.] 0x0000000000001120

Figure: Salida de perf para el programa que usa coordenadas polares.

- Finalmente, usamos los flags `-fprofile-generate`, y `-fprofile-use` para usar FDO. En promedio obtuvimos una mejora del 20%, llegando a 323K pps. Evidentemente, FDO genera información muy valiosa para el compilador. Una idea interesante (que no llegamos a probar) sería aplicar FDO nuevamente para la versión optimizada, y así ver hasta dónde se puede llegar y cuánto se puede mejorar el rendimiento.
- En cuanto a la relación tamaño-del-problema vs. rendimiento-obtenido, en el siguiente gráfico, puede observarse que, a partir de 10^4 fotones simulados, si aumentamos esta cantidad por 10 y por 100, el rendimiento se mantiene constante. Como se mencionó anteriormente, la cantidad de memoria que usa el programa no varía cuando el tamaño del problema crece. Por lo tanto, el programa no se ve obligado a tener que moverse entre los distintos niveles de la jerarquía de memoria a medida que la cantidad de fotones aumenta.



En cuanto a valores menores a 10^4 , la disminución en pps se debe probablemente a computaciones internas que realiza el programa para correr y no a nuestro algoritmo en sí, ya que para esos valores de fotones los tiempos de ejecución son valores muy pequeños. Esto puede verse al hacer profiling para una corrida que simula solo 30 fotones. Por dar un ejemplo, `do_lookup_x` está relacionado con la carga de librerías compartidas. Esto debe tomarse sólo como una posible explicación, ya que el profiling se hace para el programa completo, mientras que los pps sólo se calculan midiendo únicamente el tiempo del método `photon()`.

Samples: 25 of event 'cycles', Event count (approx.): 5394979			
Overhead	Command	Shared Object	Symbol
18,66%	tiny_mc	ld-2.31.so	[.] do_lookup_x
6,90%	tiny_mc	[kernel.kallsyms]	[k] lock_page_memcg
6,86%	tiny_mc	[kernel.kallsyms]	[k] iowrite8
6,81%	tiny_mc	libc-2.31.so	[.] __memmove_sse2_unaligned_erms
6,76%	tiny_mc	[kernel.kallsyms]	[k] crc32c_pcl_intel_update
6,70%	tiny_mc	tiny_mc	[.] main
6,64%	tiny_mc	[kernel.kallsyms]	[k] file_tty_write.isra.0
6,55%	tiny_mc	libc-2.31.so	[.] _dl_addr
6,31%	tiny_mc	ld-2.31.so	[.] check_match
5,86%	tiny_mc	[kernel.kallsyms]	[k] filemap_map_pages
5,65%	tiny_mc	ld-2.31.so	[.] mmap64
5,59%	tiny_mc	[kernel.kallsyms]	[k] __irqentry_text_end
5,48%	tiny_mc	ld-2.31.so	[.] _dl_cache_libcmp
3,71%	tiny_mc	[kernel.kallsyms]	[k] mmap_region
1,20%	tiny_mc	[kernel.kallsyms]	[k] setup_arg_pages
0,24%	perf	[kernel.kallsyms]	[k] x86_pmu_enable
0,05%	perf	[kernel.kallsyms]	[k] end_repeat_nmi
0,01%	perf	[kernel.kallsyms]	[k] intel_bts_enable_local
0,00%	perf	[kernel.kallsyms]	[k] native_write_msr

Conclusiones

- Se cambió el PRNG.
- -flt0 no nos aportó mejoras.
- Se reemplazó los dos arrays por un array de estructura.
- Se trató de evitar la generación de números aleatorios que no se van a usar.
- Se pudo demostrar que el empleo de las funciones trigonométricas son costosas.
- FDO mejora el programa en casi 20%,
- Loop unrolling y loop blocking no nos sirvió para nuestro problema.
- Pasamos de 140 K pps a 323 K pps.
- Se propone mejorar el muestreo de puntos (x,y) en el círculo de radio 1 centrado en el origen y logf.

Bibliografía

1. https://omlc.org/software/mc/Jacques2011_MonteCarlo_Welch&VanGemert.pdf
2. https://www.boost.org/doc/libs/1_48_0/doc/html/boost_random/reference.html#boost_random.reference.concepts.pseudo_random_number_generator